Call-Tree Polymorphism

Sam Estep

Abstract

...

Introduction

In any programming language, one must trade off readability and maintainability on the one hand, and performance on the other hand. One common source of this tension is the need to access objects related to a given one. Sampson (2023) illustrates this through an example interpreter of arithmetic expressions.

tree.rs

```rust
enum BinOp { Add, Sub, Mul, Div }
enum Expr {
    Binary(BinOp, Box<Expr>, Box<Expr>),
    Literal(i64),
}

impl Expr {
    fn interp(&self) -> i64 {
        match self {
            Expr::Binary(op, lhs, rhs) => {
                let lhs = lhs.interp();
                let rhs = rhs.interp();
                match op {
                    BinOp::Add => lhs.wrapping_add(rhs),
                    BinOp::Sub => lhs.wrapping_sub(rhs),
                    BinOp::Mul => lhs.wrapping_mul(rhs),
                    BinOp::Div => lhs.checked_div(rhs).unwrap_or(0),
                }
            }
            Expr::Literal(num) => *num,
        }
    }
}
```

Using `Box` is easy but leaves performance on the table. A more efficient allocation strategy is to store expressions in an *arena* as a single allocation, rather than giving each its own allocation.

tree.rs

```rust
enum Expr {
    Binary(BinOp, Box<Expr>, Box<Expr>),
    Literal(i64),
}
```

flat.rs

```rust
enum Expr {
    Binary(BinOp, ExprRef, ExprRef),
    Literal(i64),
}

#[derive(Clone, Copy)]
struct ExprRef(u32);

struct ExprPool(Vec<Expr>);
```

But this comes at a cost. We can no longer just write `expr.interp()` on some `&Expr`. Now any operation that would previously have taken an `&Expr` needs to take an `ExprRef` instead... and *also* an `ExprPool`, because the `ExprRef` doesn't have a "direct" way to access its data in the way that an

`&Expr` does. So the natural pattern is to essentially treat `ExprPool` as a monad, and implement any `Expr`-related operation as a method on `ExprPool`.

tree.rs

```
impl Expr {
```

flat.rs

```
impl ExprPool {
```

tree.rs

```
fn interp(&self) -> i64 {
    match self {
```

flat.rs

```
fn interp(&self, expr: ExprRef) -> i64 {
    match self.get(expr) {
```

tree.rs

```
let lhs = lhs.interp();
let rhs = rhs.interp();
```

flat.rs

```
let lhs = self.interp(*lhs);
let rhs = self.interp(*rhs);
```

Completed work

Theory

(syntax figure)

(static semantics figure)

(dynamic semantics figure)

Implementation

We have implemented a compiler from Moss to WebAssembly (TODO citation) (Wasm), written in Python. This implementation supports the full language described in the previous section, plus the following additional language features:

- TODO

We have also implemented a self-hosting compiler from Moss to Wasm, which can be bootstrapped from the Python implementation:

- The Python compiler is Tier 0.
- Tier 0 can be run on the Moss compiler source code to produce the Tier 1 compiler.
- The Tier 1 compiler can be run on the Moss compiler source code to produce the Tier 2 compiler.
- The Tier 2 compiler can be run on the Moss compiler source code to produce the Tier 3 compiler.

The Wasm binaries for the Tier 2 and Tier 3 compilers are bytewise identical.

Proposed work

Performance experiments

Related work

Lisp

Koka

Zig

Bibliography

Sampson, Adrian. 2023. “Flattening ASTs (and Other Compiler Data Structures).” May 1. https://www.cs.cornell.edu/~asampson/blog/flattening.html.